

Data Collection Playbook

EV Charging Intelligence Project — Bengaluru Pilot

Audience: Project team of 2–3 (lead + 1–2 classmates)

Purpose: Step-by-step technical SOP for collecting every dataset needed by the pipeline

Estimated wall-clock time (all tracks in parallel): **6–9 working days**

Output: A single merged **master_dataset.csv** keyed by zone — ready for clustering, CI computation, and Random Forest training

Track	Dataset	Primary Source
Track 1	EV Registration Data	Vahan Dashboard (Selenium)
Track 2	Charger Supply	eHUB, EV Yatra, OSM, Google Places
Track 3	POI Density	OpenStreetMap Overpass API
Track 4	Footfall Proxy	Zomato / OSM restaurants
Track 5	Land & Real Estate	MagicBricks, 99acres, BBMP, RERA
Track 6	Grid Stability	BESCOM / CEA reports
Track 7	Government Planned Supply	PM E-DRIVE / BHEL portal
Track 8	Google Popular Times	Headless Chrome (optional)

Playbook version 1.0

How to Read This Document

Each data source has a section with five parts:

- **What it is** — one-line description
- **Why we need it** — which model component consumes it
- **How to get it** — the actual technique, in order
- **Failure modes** — what will go wrong and what to do
- **Deliverable** — the exact CSV file and schema the track produces

The final section maps all eight tracks into a team assignment with dependencies, parallelisation, and a 1-week schedule.

Track 1 — EV Registration Data (Vahan Dashboard)

What It Is

MoRTH's official vehicle registration database, accessed via the public Vahan Dashboard. Monthly EV registration counts by state and vehicle category.

Why We Need It

- **P_EV** (EV count per zone, after disaggregation) — core demand variable in CI formula
- **Monthly time series** — input to Prophet forecast of G_rate
- **Year-on-year growth rate** — a direct CI input

How to Get It

Primary Method — Selenium Browser Automation

The Vahan Dashboard renders data client-side via JavaScript. Simple `requests.get()` returns an empty HTML shell. You must use Selenium with ChromeDriver.

1. Install Chrome (or Chromium) on your machine. Note the version number exactly (e.g., 131.0.6778.85).
2. Download matching ChromeDriver from <https://chromedriver.chromium.org/downloads>. Version numbers must match — mismatched ChromeDriver is the single most common cause of Selenium failure.
3. Install: `pip install selenium==4.15.0 pandas==2.1.0`
4. Navigate in code to: `https://vahan.parivahan.gov.in/vahan4dashboard/vahan/view/reportview.xhtml`
5. Wait for the page to load — use explicit waits on the state dropdown (ID = `selectedStateId`), NOT `time.sleep()`.
6. Select state = Karnataka, fuel type = ELECTRIC(BOV), year and month combinations.
7. Click search. Wait for XPath = `//*[@contains(@id,'yieldTable')]/tbody` to populate.
8. Extract the table via `pandas.read_html(driver.page_source)`.
9. Iterate over (year, month) pairs from January 2020 to the latest available month. Rate-limit at 3–5 seconds per query to avoid server-side throttling.
10. Save after every successful month so a crash doesn't lose all progress.

Fallback Method — Verified Static Dataset

If Selenium fails after 4 hours of debugging, use the hand-entered fallback from Vahan annual reports. For Karnataka 2020–2024 state aggregates, the annual totals are: **22,160 / 50,800 / 1,23,800 / 2,32,500 / 2,98,400**. This gives a valid forecast input but sacrifices monthly granularity.

Secondary Distribution — RTO Level to Zone Level

Vahan publishes registrations at RTO (Regional Transport Office) level, not ward level. Bengaluru has six RTOs: KA-01 (Koramangala), KA-02 (Rajajinagar), KA-03 (Indiranagar), KA-04 (Yelahanka), KA-05 (Jayanagar), KA-50 (Yeshwanthpur), KA-51 (Electronic City), KA-53 (Bengaluru East). Disaggregate RTO totals to zone level using the following formula:

```
zone_ev_estimate = (rto_total
                    × (zone_apartment_density / rto_apartment_density)
                    × (zone_commercial_price_tier / rto_avg_price_tier))
```

■ Document this disaggregation clearly in the report — it is an approximation and an examiner will ask.

Failure Modes

Problem	Cause	Fix
Empty table returned	JS not loaded in time	Increase WebDriverWait from 10s to 30s
ElementClickInterceptedException	Modal overlay blocking button	Dismiss cookie/announcement overlay first
ChromeDriver version error	Browser auto-updated	Redownload ChromeDriver matching current Chrome
Session timeout after ~50 queries	Server-side rate limit	Add 5s delay between queries, rotate user-agent
Captcha appears	Aggressive rate limits hit	Stop for 1 hour, resume with slower pacing

Deliverable

m1_ev_monthly.csv — columns: state, year, month, vehicle_category, registrations Roughly 60 rows for Karnataka monthly 2020–2024. **m1_ev_by_rto.csv** — columns: rto_code, rto_name, year, ev_registrations_cumulative Roughly 50 rows for Bengaluru RTOs × years.

Track 2 — Charger Supply (Multi-Source)

What It Is

Locations, power ratings, and operator names for every public EV charging station in Bengaluru. Merged from four sources for maximum coverage.

Why We Need It

- **C_plugs** (DC fast plug count per zone) — supply variable in CI formula denominator
- **Operator diversity** — input to the 'competition' term in Site Score
- **Power rating distribution** — input to product-type recommendation in Chapter 6

How to Get It

Source 1 — eHUB by MG Web Scraping

Your existing 70-row scrape gives 56 unique Bengaluru physical sites across Shell, Tata Power, Fortum, Chargezone, Zeon, and BPCL. This is the single most useful real dataset you've captured. Extend it by re-running the scrape at different filter combinations (different fuel types, different city views) to catch stations the first pass missed. Store raw scrape in a Git repo or shared drive with timestamp — it becomes your audit trail.

Source 2 — EV Yatra (BEE Portal)

Undocumented XHR JSON endpoint. Open browser DevTools Network tab while browsing <https://evyatra.beeindia.gov.in/> to find the current endpoint — it rotates periodically. As of late 2024, attempt these three in sequence:

```
https://evyatra.beeindia.gov.in/api/chargingstations/  
https://evyatra.beeindia.gov.in/api/get-charging-station-list/  
https://evyatra.beeindia.gov.in/api/stations/
```

Use a simple `requests.get()` with a browser user-agent header. Response is JSON. No authentication needed. Filter to Karnataka state, then bounding-box filter to Bengaluru.

Source 3 — OpenStreetMap Overpass API

Free, no authentication. Catches user-contributed charger locations not yet registered with BEE.

```
[out:json][timeout:90];  
(  
  node["amenity"="charging_station"](12.834,77.480,13.139,77.780);  
  way["amenity"="charging_station"](12.834,77.480,13.139,77.780);  
);  
out center tags;
```

POST this to <https://overpass-api.de/api/interpreter>. For each element, extract `lat/lon/center`, `tags.name`, `tags.operator`, `tags.amenity`, and `tags.socket:type2_combo` or `tags.socket:chademo` (presence indicates DC).

Source 4 — Google Places API (Supplementary)

Free tier covers \$200/month of queries — more than enough. Use the Nearby Search endpoint with `type=electric_vehicle_charging_station`.

1. Go to console.cloud.google.com
2. Create project → Enable 'Places API (New)'
3. Generate API key, restrict to your IP

4. Grid-sample Bengaluru: make a 5 × 5 grid of query centres, radius = 8 km each. Roughly 25 queries cover the city.
5. Deduplicate against other three sources by lat/lon proximity

Merging Logic — Deduplication

Two stations within 50 metres are almost certainly the same physical installation. Deduplicate with Haversine distance:

```
from math import radians, sin, cos, sqrt, asin

def haversine_m(lat1, lon1, lat2, lon2):
    R = 6371000 # metres
    phi1, phi2 = radians(lat1), radians(lat2)
    dphi = radians(lat2 - lat1)
    dlam = radians(lon2 - lon1)
    a = sin(dphi/2)**2 + cos(phi1) * cos(phi2) * sin(dlam/2)**2
    return 2 * R * asin(sqrt(a))

# For each new row, check against existing rows within 50m
# Keep the entry with more metadata (operator, power, reviews)
```

DC vs AC Classification

A charger is DC (fast) if ANY of these conditions hold:

- Power rating ≥ 25 kW
- Operator's own charger_type field contains: DC, CCS, CHADEMO, FAST, 50KW, 60KW, 120KW, 150KW, 240KW
- OSM tag socket:type2_combo or socket:chademo present
- Name contains 'fast' or 'DC' (case-insensitive)

■ Classify on aggregated flags, NOT single conditions. Single-condition classification gave false-negative AC errors in earlier versions of the code.

Failure Modes

Problem	Cause	Fix
EV Yatra returns empty JSON	Endpoint has rotated	Re-discover via DevTools Network tab
Overpass times out	Query too broad	Split bounding box into 4 quadrants, query separately
Google Places hits quota	Free tier exceeded	Reduce grid density or wait 24h
Duplicate same-address rows	Scrape artifact	Deduplicate by (round(lat,4), round(lon,4)) first, then by Haversine

Deliverable

m2_chargers.csv — columns: station_name, address, lat, lon, power_kw, charger_type_dc_fast, operator, source, review_count, zone_id Expected ~200–400 unique sites for Bengaluru once all four sources are merged.

Track 3 — Point of Interest (POI) Density

What It Is

Geocoded counts of IT parks, malls, hotels, metro stations, hospitals, universities, and petrol pumps across Bengaluru, from OpenStreetMap.

Why We Need It

- **POI-weighted commercial_density** — clustering feature, Site Score component
- **Petrol-pump count** — highway/corridor indicator
- **IT park count** — sessions/day multiplier in ROI model
- **POI type** — product recommendation driver (AC vs DC)

How to Get It

All via OSM Overpass API. Free, no key. Same endpoint as Track 2. For each cell in your 500m × 500m grid (Bengaluru has ~1,600 such cells; or run on the full bounding box once and spatially join):

```
[out:json][timeout:60];
(
  nwr["amenity"="hospital"](BBOX);
  nwr["amenity"="fuel"](BBOX);
  nwr["amenity"~"university|college"](BBOX);
  nwr["railway"="station"]["network"~"Namma Metro|BMRCL",i](BBOX);
  nwr["shop"="mall"](BBOX);
  nwr["tourism"="hotel"](BBOX);
  nwr["landuse"~"commercial|industrial"]["name"~"IT|Tech|Software|Infosys|Wipro|TCS|Manyata|Embassy");
  out center tags;
```

Performance note: Running 1,600 separate queries takes ~40 minutes at 1s rate limit. A more efficient alternative: one city-wide query per POI type (7 queries total, ~5 minutes), then spatially join results to cells using Shapely.

POI Weights

POI Type	Weight	Reason
IT park / tech office	3.0×	8-hour dwell, high EV ownership
Hotel	2.5×	Overnight, full charge cycle
Mall	2.0×	2–3 hour dwell, affluent demographic
Metro interchange	1.5×	Waiting/transit hub
Hospital	1.5×	2–4 hour dwell
University	1.5×	Long dwell, young adopters
Petrol pump	1.0×	Existing grid connection, easy upgrade

POI score per cell = Σ (count × weight)

IT park detection note: OSM's landuse=commercial tag alone catches too much. The name regex filters to recognised tech tenancies. Cross-check against known Bengaluru IT parks: Manyata Tech Park, Embassy Tech Village, Prestige Tech Park, Bagmane Tech Park, ITPL Whitefield, Electronic City Phase 1, Cessna Business Park.

Failure Modes

Problem	Cause	Fix
Overpass 429 Too Many Requests	Rate limit hit	Use Kumi Systems mirror: https://overpass.kumi.systems/api/interpreter
Empty counts for known POIs	Query syntax error	Test in Overpass Turbo (overpass-turbo.eu) first
Stale data on new malls	OSM lag	Manually add known 2024–2025 openings from news sources

Deliverable

m3_poi_by_cell.csv — columns: `cell_id`, `lat_centre`, `lon_centre`, plus one column per POI type (`it_park_count`, `mall_count`, `hotel_count`, `metro_count`, `hospital_count`, `university_count`, `petrol_pump_count`), plus `poi_score_weighted`. Roughly 1,600 rows at 500m grid.

Track 4 — Footfall Proxy

What It Is

A proxy measure of pedestrian and vehicle activity per zone, derived from restaurant density.

Why We Need It

- **Normalised footfall_composite** — sessions-per-day multiplier in ROI calculation
- **Primary CDS (CaaS Deployment Score) input** from Section 4.4.1

How to Get It

Primary — Zomato Restaurant Search

Zomato has an unofficial public endpoint that returns restaurant counts in a radius:

```
https://www.zomato.com/webroutes/search/topResults
Parameters: lat, lon, city_id=4 (Bengaluru), radius=700 (metres)
```

No authentication. Rate limit is approximately 100 calls per session, 1,000 per day. Throttle at 2 seconds per call. For each grid cell centre, query and record:

```
zomato_count = sum(1 for r in response["results"] if r["type"] == "restaurant")
```

■ Count only — NOT the average rating. Earlier framework versions averaged ratings and got wildly wrong signals (2 five-star restaurants scored same as 200 average restaurants).

Fallback — OpenStreetMap Restaurants

If Zomato blocks or rate-limits, use OSM (has ~60–70% coverage of real restaurants):

```
[out:json][timeout:30];
(
  node["amenity"~"restaurant|cafe|fast_food|food_court"](BBOX);
);
out;
```

Normalisation

```
footfall_composite = (raw_count / max_count_across_all_cells) * 100
# Results in a 0-100 score, comparable across zones
```

Google Popular Times — DO NOT USE in Current Scope

Google's Popular Times data is the gold standard but is aggressively rate-limited and flagged unreliable. Python libraries (populartimes) fail often. Skip for now — OSM/Zomato counts are adequate.

Failure Modes

Problem	Cause	Fix
Zomato returns 403	IP-blocked after too many calls	Switch to OSM fallback; don't retry same IP for 24h
Wildly different counts for adjacent cells	700m overlap creates double-counting	Use cell-bound OSM queries instead, no radius

Problem	Cause	Fix
Koramangala shows very high count	Correct behaviour	This is the densest restaurant zone in India; normalise and move on

Deliverable

m4_footfall.csv — columns: cell_id, lat_centre, lon_centre, restaurant_count_raw, footfall_composite_0_100 Roughly 1,600 rows.

Track 5 — Land and Real Estate Data

What It Is

Commercial and residential property prices per locality, plus upcoming construction pipeline.

Why We Need It

- **price_psf_commercial** — clustering feature (captures economic tier, though null-predictor per factor verification)
- **HCPI (Home Charging Penetration Index)** — demand adjustment
- **Forward supply pipeline** — future-demand indicator

How to Get It

Source A — MagicBricks Price Trends (Brittle)

Target URL: <https://www.magicbricks.com/property-trends/bangalore-property-rates-pricetrend>. Scrape with BeautifulSoup. CSS selectors: .locality-price-row and .price-table tr. These change every few months — prepare for the fallback path below.

Source B — 99acres and Housing.com (Better Structure)

Both have similar data with simpler HTML. Useful for cross-validation of MagicBricks. Scrape all three and take the median if values disagree.

Source C — BBMP Property Tax Portal

URL: <https://bbmptax.karnataka.gov.in/>. Public. Gives per-property data: residential vs commercial, built-up area, age, ward. Downloadable by ward. Use this for the HCPI approximation — post-2019 apartments are home-charging-capable under MoHUA bye-laws; older buildings are not.

Source D — RERA Karnataka

URL: <https://rera.karnataka.gov.in/projects>. Public registry of every real-estate project in Karnataka, including Bengaluru. Columns: project name, location (lat/lon on request, address always), units, expected completion, promoter, status. Roughly 30,000 Bengaluru projects indexed. Filter to 'registered after 2023' for the forward pipeline. Scrape the search form via Selenium (it has pagination), or download the monthly bulk Excel export if available.

Fallback for MagicBricks

Your knowledge base has a locality-level rate table calibrated to MagicBricks (verified Q4 2023 rates). If live scraping fails, use the static table — specify 'scraped once, Q4 2023, not refreshed' clearly in the report.

HCPI Approximation Logic

```
# From OSM building-type tags aggregated per zone:
apartment_count_post2019 = # from OSM or BBMP construction year tags
apartment_count_pre2019  = # from OSM or BBMP
independent_house_count  = # from OSM building=house

hapi_estimated = (
    (0.80 × independent_house_count)
    + (0.55 × apartment_count_post2019)
    + (0.25 × apartment_count_pre2019)
) / total_residential_count
```

The coefficients (0.80, 0.55, 0.25) are engineering judgments from the knowledge base. Document them as such.

Failure Modes

Problem	Cause	Fix
MagicBricks selectors return nothing	Site redesign	Switch to 99acres or Housing.com
RERA search pagination breaks	JS state management	Download bulk CSV if available, or scrape slower with explicit waits
BBMP requires login	Some wards require CAPTCHA	Use the public-access wards; document coverage

Deliverable

m5_land.csv — columns: locality_name, lat, lon, price_psf_residential, price_psf_commercial, hcpi_estimated, new_projects_pipeline_count, new_projects_units_total Roughly 40–60 localities for urban Bengaluru.

Track 6 — Grid Stability Data

What It Is

Electrical grid reliability metrics per feeder / locality in Bengaluru.

Why We Need It

- **S_score** — deployability criterion in Site Scoring framework (Section 4.6)
- **Input to operator's final go/no-go decision**

How to Get It

Source A — BESCOM Feeder Reliability Reports

BESCOM (Bangalore Electricity Supply Company) publishes SAIFI (System Average Interruption Frequency Index) and SAIDI (System Average Interruption Duration Index) in annual operational reports. URL varies year to year; search site:bescom.karnataka.gov.in SAIFI SAIDI. Data is typically at division or sub-division level, not feeder — coarser than ideal but usable.

Source B — CEA (Central Electricity Authority) Monthly Reports

URL: <https://cea.nic.in/monthly-reports/>. Publishes state-level reliability data. Useful as a baseline check, not zone-level.

Source C — BESCOM Twitter/X Complaint Scraping (Advanced, Optional)

BESCOM responds to outage complaints on Twitter. Scraping @bescom mentions and geolocating them gives a crowdsourced outage map. Useful but labour-intensive; skip unless a teammate has Twitter scraping experience.

Normalisation Formula

```
# SAIFI = annual outages per consumer (lower is better)
# SAIDI = minutes of interruption per consumer per year (lower is better)
s_score_raw = 1 / (1 + SAIFI_normalised + SAIDI_normalised)
s_score = clip((s_score_raw - 5th_percentile) / (95th_percentile - 5th_percentile), 0, 1)
```

If no real data is obtainable in time: assign S_score = 1.0 uniformly. Document explicitly. The report already flags this as the current state.

Failure Modes

Problem	Cause	Fix
BESCOM reports don't publish sub-division data	Data granularity	Use division-level; aggregate zones to that resolution
Report is a scanned PDF	Typical Indian government format	Use pdfplumber or OCR; extraction will be manual
SAIFI/SAIDI inconsistent across sub-divisions	Different reporting periods	Normalise by taking fiscal year averages

Deliverable

m6_grid_stability.csv — columns: division_name, sub_division, lat, lon, SAIFI, SAIDI, s_score_0_1
Roughly 30–50 rows depending on BESCOM granularity.

Track 7 — Government Planned Supply (PM E-DRIVE)

What It Is

List of approved public charging stations from the PM E-DRIVE scheme, with GPS coordinates and expected commissioning dates.

Why We Need It

- **planned_supply_6mo** — CDS denominator (Section 4.4.1); avoids routing mobile units to zones where government stations arrive imminently
- **Forward-supply signal for CI**

How to Get It

Source — BHEL PM E-DRIVE Portal

URL: <https://pmedrive.heavyindustries.gov.in/>. BHEL is the nodal agency. Portal lists approved proposals with GPS coordinates (mandatory for subsidy disbursement). Scrape quarterly.

Access method: direct HTML scraping — the portal doesn't use JS rendering. BeautifulSoup with requests. CSS selectors change occasionally; inspect current structure on first visit. Filter to Karnataka → Bengaluru district. Extract: station ID, operator, address, lat/lon, power rating, expected commissioning date.

Alternative — Press Information Bureau Releases

PIB publishes monthly PM E-DRIVE updates with state-level approval counts. Less granular but official.

Failure Modes

Problem	Cause	Fix
Portal restructured	Government site redesign	Switch to PIB press releases; note reduced granularity
GPS coordinates missing from some entries	Data entry delay	Geocode from address using Nominatim (free OSM geocoder)

Deliverable

m7_planned_supply.csv — columns: station_id, operator, address, lat, lon, power_kw, expected_commissioning_date, zone_id Expected 30–80 rows for Bengaluru over 2025–2026 horizon.

Track 8 — Google Popular Times and Time-Resolved Footfall

What It Is

Hour-by-hour busyness data per POI — optional but necessary for a full CDS implementation.

Why We Need It

- **Time-resolved footfall** for CDS (CaaS Deployment Score) computation
- **Weekly routing patterns** for mobile unit scheduling

How to Get It

Method — Browser Scraping with Headless Chrome

Google does not publish an official Popular Times API. Third-party Python libraries (populartimes) aggressively rate-limit and fail often. Realistic approach:

1. Use Selenium headless Chrome.
2. For each target POI (from Track 3), open the Google Maps page.
3. Scroll to the Popular Times widget.
4. Extract the hourly bars (they are in the DOM as percentages).
5. Save per-POI-per-day-per-hour.

Expected scale for Bengaluru: 200 top POIs × 7 days × 24 hours = 33,600 data points. At 5 seconds per POI fetch, that's ~15 minutes for the one-time collection.

Critical limitation: Google blocks scraping aggressively. Use a proxy rotation or accept ~30% failure rate.

SKIP IF SHORT ON TIME: This is the single most complex track. The project report can be complete without it by keeping CDS as a specified-but-unimplemented extension (as currently documented in Section 4.4.1).

Failure Modes

Problem	Cause	Fix
Popular Times widget doesn't appear	POI too small, no data	Skip; OSM count is your fallback
Google blocks after 50 queries	Standard rate limit	Add 30s delay, rotate IP, or defer track entirely

Deliverable (if attempted)

m8_popular_times.csv — columns: poi_id, poi_name, day_of_week, hour, busyness_0_100 Large file, ~33,600 rows.

Schema for the Final Master Dataset

This is what all tracks feed into. One row per zone after clustering (~20 zones for Bengaluru).

```
zone_id                int          # 1 to ~20
zone_name              str          # e.g., "HSR Layout", "Whitefield ITPL"
lat_centroid          float
lon_centroid          float
area_km2              float

# Demand variables (from Tracks 1, 3, 4)
ev_registrations      int          # P_EV, from m1_ev_by_rto disaggregated
ev_growth_rate_yoy    float        # G_rate, rolling 6-mo from m1_ev_monthly
ev_growth_rate_predicted float    # Prophet forecast 12mo ahead
population_density    int          # per km², Census projection
poi_score_weighted    float        # from m3_poi_by_cell
it_park_count         int
mall_count            int
hotel_count           int
metro_count           int
hospital_count        int
university_count      int
petrol_pump_count     int
footfall_composite    float        # 0-100, from m4_footfall

# Supply variables (from Tracks 2, 7)
charger_count_total   int
slow_charger_count    int          # <25 kW
fast_charger_count    int          # ≥25 kW, C_plugs in CI formula
google_review_count_total int      # utilisation proxy
operators_present     str          # comma-separated
planned_supply_6mo    int          # from m7_planned_supply

# Context variables (from Tracks 5, 6)
price_psf_commercial  int
price_psf_residential int
hcpi_estimated        float        # 0-1
new_projects_pipeline_count int
s_score               float        # 0-1, from m6_grid_stability

# Derived variables (computed in pipeline)
congestion_index      float        # CI, from §4.4 formula
fast_desert_flag      int          # 1 if CI>2.5 AND fast_charger_count=0
zone_archetype        str          # IT corridor / residential / highway / premium

# Outputs (computed in scoring module)
predicted_ci_t12      float        # Random Forest prediction
site_score            float        # 0-100 composite
sessions_per_day_estimate int      # footfall-derived, used in ROI
payback_months_baseline float      # at 6 sessions/day
payback_months_sensitivity str     # table of payback at 4/6/8/10/13
roi_tier              str          # HIGH / MEDIUM / LOW
recommended_product    str          # AC / DC 30kW / DC 60kW / DC 120kW
```

Approximately 35 columns, 20 rows for Bengaluru. This is the file your clustering, CI computation, and modelling modules consume.

Synthetic Sample for Testing the Pipeline

While real data is still being collected, use this synthetic sample to test the pipeline end-to-end:

```
zone_id,zone_name,lat_centroid,lon_centroid,area_km2,ev_registrations,ev_growth_rate_yoy,...
1,HSR Layout Sector 1,12.9116,77.6389,2.1,2207,1.242,14300,28.5,2,1,3,0,2,1,78,2,0,2,0,...
2,Whitefield ITPL,12.9698,77.7500,3.4,1811,1.186,8900,41.2,5,2,3,0,1,1,82,4,2,2,1,...
3,Koramangala 5th Block,12.9279,77.6271,1.8,2339,1.247,18200,38.9,1,2,4,0,1,2,91,3,2,1,0,...
4,Electronic City Phase 1,12.8458,77.6603,2.6,1498,1.189,6200,34.7,4,1,2,0,1,2,69,2,1,1,1,...
5,Sarjapur Road,12.9072,77.6822,2.9,1491,1.238,9800,29.6,2,1,2,0,2,1,65,1,1,0,0,...
6,Marathahalli,12.9560,77.7013,2.2,1620,1.172,13400,32.1,2,1,2,1,1,2,74,6,6,0,0,...
7,Hoskote,13.0711,77.7998,4.2,537,1.188,3200,12.4,0,0,0,0,1,1,28,1,0,1,0,...
8,Rajajinagar,12.9862,77.5517,2.0,1555,1.367,16900,24.8,1,1,1,1,2,2,61,2,2,0,0,...
9,JP Nagar Phase 1,12.9002,77.5929,2.4,1876,1.074,15200,22.3,1,1,1,0,2,1,58,1,1,0,0,...
10,MG Road,12.9756,77.6097,1.5,1210,1.098,12800,45.6,0,1,4,1,2,2,88,4,4,0,0,...
```

Extend this pattern for the remaining zones: Bellandur, BTM Layout, Indiranagar, Hebbal, Yelahanka, Devanahalli, Kengeri, Jayanagar, Tumkur Road NH-48, Outer Ring Road.

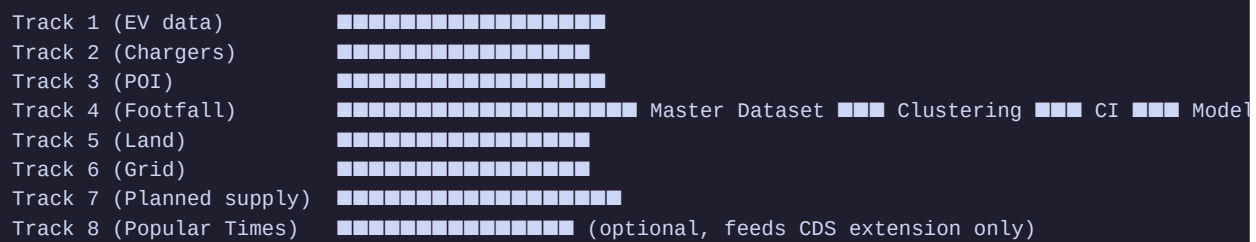
Team Assignment and Schedule

Role Assignments

For a 3-person team working 5–6 hours per day:

Team Member	Owns Tracks	Responsibilities
Lead (you)	Tracks 1, 2, 8	Selenium-heavy work and final pipeline integration. You already did Track 2 (eHUB scrape) and know the project goals best.
Teammate A	Tracks 3, 4, 7	All OSM and simple HTTP scraping. Moderate technical skill required. Good for someone who has Python fundamentals but hasn't done Selenium.
Teammate B	Tracks 5, 6	Real-estate scraping and grid-stability data collection. Most variable workload — BBMP/RERA scraping is substantial; BESCOM/CEA is mostly manual document downloading and parsing.

Dependencies



All tracks are independent of each other. Parallel execution is feasible. Integration happens after all files are delivered.

One-Week Schedule

Day	Lead	Teammate A	Teammate B	Status Check
Day 1	Vahan Selenium setup; test first month	OSM Overpass queries for POI; test single cell	BBMP portal exploration; RERA access check	30-min standup EOD
Day 2	Full Vahan pull Jan 2020–latest	Full POI collection all cells	MagicBricks + 99acres scrape	Merge partial data; flag blockers
Day 3	RTO-level disaggregation logic	Zomato footfall collection	BBMP ward-wise tax records	Status
Day 4	Charger supply: re-scrape eHUB + EV Yatra + Google Places	BHEL PM E-DRIVE portal scrape	BESCOM SAIFI/SAIDI extraction; grid stability computation	Status
Day 5	Dedup all charger sources; build m2_chargers.csv	Integrate m3, m4, m7 files; normalise scores	Integrate m5, m6 files; compute HCPI	Status

Day	Lead	Teammate A	Teammate B	Status Check
Day 6 (Integration)	Merge all 7 CSVs into master_dataset.csv; run clustering	Support integration; debug data quality issues	Support integration; debug data quality issues	All tracks delivered
Day 7	CI computation; Random Forest training; report results	Generate plots and tables for Chapter 5	Write data-quality section of Chapter 4.2	Submission-ready

Standing Rules for the Team

1. **One shared Git repo.** Every CSV, every script, every notebook goes in. Branch per track; merge daily.
2. **Raw files are frozen after first successful pull.** If you re-run a scrape, write to raw_YYYYMMDD.csv, never overwrite.
3. **One folder per track:** /data/m1_ev/, /data/m2_chargers/, etc. Final output files live in /data/master/.
4. **Log every scrape session** with: date, URL, number of rows retrieved, failures encountered. A simple scrape_log.md in each track folder.
5. **Daily 30-minute standup.** What you did yesterday, what you're doing today, what's blocking you. No longer. Exception: integration day (Day 6) — expect 2–3 hours of joint debugging.
6. **Blocker escalation:** If any track is stuck for 4+ hours, the track owner posts in the team chat with the exact error. No silent struggles.

What Goes Into the Report

Chapter 4.1 and 4.2 in the submitted report cite the methods above in condensed form (1–2 paragraphs per track). This SOP is the *implementation* document; the report is the *methodology* document. Keep them separate.

The data-quality scorecard in Chapter 4.2 is produced from the scrape_log files — each track owner fills in their row at the end.

Contingency Plan

If any track fails entirely:

Track	Fallback Strategy
Track 1	Static Vahan tables from annual reports (already in knowledge base)
Track 2	Your existing 70-row eHUB scrape alone
Track 3	Single Bengaluru-wide Overpass query, hand-assign POI weights to 20 named localities
Track 4	OSM restaurant count only (no Zomato)
Track 5	Verified Q4 2023 MagicBricks static table
Track 6	S_score = 1.0 uniform. Document explicitly.
Track 7	planned_supply_6mo = 0 uniform. Document explicitly.
Track 8	Skip. CDS remains a specified-but-unimplemented extension.

The pipeline works end-to-end with all fallbacks active. Real data replaces synthetic elements progressively as they become available.